

INSTITUT TEKNOLOGI SEPULUH NOPEMBER

Faculty of Intelligent Electrical, Electronic, and Informatics Engineering

Department of Information Systems

Bachelor of Computer Science in Information Systems Program

ES234211 – Programming Fundamental

Small Project Guideline

Simple Game Application with Java Swing GUI, Login, Statistics, and Database

Course Convenor : Ahmad Muklason

Project Type : Individual Project

Submission Deadline : Friday, 26 June 2026

Submission Media : GitHub Link and YouTube Video Link

Contents

1	Project Overview	1
2	Learning Objectives	1
3	General Project Description	1
4	Important Update: GUI Using Java Swing	2
4.1	What Students Must Do	2
4.2	What Students Should Not Do	2
4.3	Expected GUI Windows	3
5	Project Requirements	3
5.1	Functional Requirements	3
5.2	Non-Functional Requirements	4
6	Recommended Game: Tic-Tac-Toe	5
6.1	Basic Rules	5
6.2	Swing-Based Board	5
6.3	Suggested Score Calculation	5
7	Database Requirement	5
7.1	Database Management System	5
7.2	One Table Only Rule	6
7.3	Example Database Schema	6
7.4	Example SQL for MySQL	6
7.5	Example SQL for PostgreSQL	7
7.6	Example SQL for SQL Server	7
8	Suggested Class Structure	8
8.1	Simple OOP Rules	8
9	Program Flow	9
10	Starter Code and Completion Clues	9
10.1	Main Class	10
10.2	DatabaseManager Class	10
10.3	Player Class	11
10.4	PlayerService Login Method	12
10.5	Updating Statistics	13
10.6	LoginFrame Event Handling	13
10.7	MainMenuFrame Button Handling	14
10.8	GameLogic Class	15

10.9 GameFrame Button Handling	17
10.10TopScorersFrame Using JTable	18
10.11Top 5 Query	19
11 Parts That Must Be Modified or Completed by Students	19
12 Minimum Code Requirements	20
13 Development Steps	21
14 Testing Guide	21
14.1 GUI Testing	22
14.2 Game Testing	22
14.3 Database Testing	22
15 GitHub Submission Requirement	22
15.1 Recommended GitHub Repository Structure	23
15.2 README File Content	23
16 YouTube Video Requirement	25
16.1 Recommended Video Structure	26
17 Classroom Explanation	26
18 Submission	27
18.1 Submission Deadline	27
18.2 Submission Items	27
18.3 Submission Format	27
19 Assessment Criteria	27
20 Academic Integrity	28
21 Common Mistakes to Avoid	29
22 Final Checklist	29
23 Closing Statement	30

1. Project Overview

In this small project, each student is required to complete a simple game application using **Java, Java Swing GUI, simple Object-Oriented Programming**, and **one-table database storage**. The game can be similar to Tic-Tac-Toe or another simple game with comparable complexity.

The graphical user interface code using **Java Swing** is already provided by the lecturer. Students are not expected to design the GUI from zero. Instead, students are required to **modify, complete, connect, and test** several parts of the provided program.

The application must require the user to log in, allow the user to play a game, record game statistics, and display the Top 5 scorers. The statistics must be saved in a relational database using only one table.

This project is an **individual project**. Each student must submit and explain their own work.

2. Learning Objectives

After completing this project, students are expected to be able to:

1. Apply programming fundamentals in a small but complete application.
2. Use simple Object-Oriented Programming by dividing the program into classes.
3. Understand the basic role of model, service, logic, and user interface classes.
4. Complete Java Swing GUI event handling using buttons and input fields.
5. Connect a Java program to MySQL, PostgreSQL, or SQL Server using JDBC.
6. Read and update player data from a database table.
7. Implement game logic, including win, lose, draw, and score calculation.
8. Record player statistics such as wins, losses, draws, and score.
9. Display the Top 5 scorers from the database.
10. Upload source code to GitHub and explain the project through a YouTube video.

3. General Project Description

Students must complete a simple game application with a graphical user interface. The recommended game is **Tic-Tac-Toe**. However, students may use another simple game if the main features are still available: login, play game, update statistics, and show Top 5 scorers.

The application must use **Java Swing**. Console-only applications are not accepted for this updated version of the project. The user must interact with the program through windows, buttons, labels, text fields, password fields, tables, and dialog boxes.

The GUI code is already provided. Students must focus on completing the missing program logic and connecting the GUI to the database and game logic. The project should remain simple and suitable for Programming Fundamental students.

The database must contain **one table only**. The schema is already given. Students must follow the given schema and must not create additional tables unless explicitly allowed by the lecturer.

4. Important Update: GUI Using Java Swing

This project must now be implemented using **Java Swing GUI**. The lecturer has already provided the starter GUI code. Students must modify or complete the required parts of the program.

4.1. What Students Must Do

Students are required to complete the following parts:

1. Complete the database connection settings.
2. Complete the login checking process.
3. Complete the event handling code for GUI buttons.
4. Complete the game logic, including valid move checking and winner checking.
5. Connect the Game GUI with the game logic class.
6. Update player statistics after the game ends.
7. Display personal statistics in the GUI.
8. Display the Top 5 scorers in the GUI table.
9. Make sure the application can run from the main class.
10. Upload the completed project to GitHub.

4.2. What Students Should Not Do

Students are not allowed to:

1. Replace the Swing GUI with a console-only program.
2. Replace the provided GUI with an unrelated interface.

3. Use inheritance, interface, abstract class, or advanced OOP concepts.
4. Create more than one database table.
5. Hardcode Top 5 scorer data inside the Java program.
6. Submit a program that does not connect to the database.
7. Submit code that they cannot explain.

4.3. Expected GUI Windows

The provided GUI may contain the following windows or panels:

1. Login Window.
2. Main Menu Window.
3. Game Window.
4. My Statistics Window.
5. Top 5 Scorers Window.

The exact layout may follow the GUI code provided by the lecturer. Students may make minor visual improvements, but the required features must remain clear and functional.

5. Project Requirements

5.1. Functional Requirements

The application must provide the following features:

1. Login Feature

- (a) The user enters username and password in the Swing Login Window.
- (b) The program checks username and password from the database.
- (c) If login succeeds, the program opens the Main Menu Window.
- (d) If login fails, the program shows an error message using `JOptionPane`.

2. Game Feature

- (a) The user can start a new game from the Main Menu Window.
- (b) The game is displayed using Swing components, such as `JButton`, `JLabel`, and `JPanel`.
- (c) The program must detect win, lose, or draw.

- (d) The program must show the final result after the game ends.
- (e) The program must prevent invalid moves.

3. Statistics Feature

- (a) The program records the number of wins.
- (b) The program records the number of losses.
- (c) The program records the number of draws, if applicable.
- (d) The program records the total score.
- (e) The program updates the database after each completed game.

4. Top 5 Scorer Feature

- (a) The program displays the Top 5 players based on score.
- (b) If two players have the same score, the player with more wins should be ranked higher.
- (c) The Top 5 data must be retrieved from the database.
- (d) The Top 5 data should be displayed using a Swing table such as `JTable`.

5. Navigation Feature

- (a) The user can move from Login Window to Main Menu Window.
- (b) The user can open Game Window, My Statistics Window, and Top 5 Scorers Window from the Main Menu Window.
- (c) The user can return to the Main Menu Window after finishing an activity.
- (d) The user can exit the application safely.

5.2. Non-Functional Requirements

The application must satisfy the following requirements:

1. The program must be written using Java.
2. The program must use Java Swing for the GUI.
3. The program must apply simple Object-Oriented Programming.
4. The program must be divided into several classes.
5. The program must not use inheritance, interface, abstract class, or advanced OOP concepts.
6. The program must connect to MySQL, PostgreSQL, or SQL Server.
7. The database must contain only one table.

8. The source code must be uploaded to GitHub.
9. The demonstration video must be uploaded to YouTube.

6. Recommended Game: Tic-Tac-Toe

6.1. Basic Rules

Tic-Tac-Toe is played on a 3×3 board. The player uses the symbol X, while the computer uses the symbol O. The player and computer take turns placing their symbols on the board.

A player wins if they place three symbols in the same row, column, or diagonal. If all cells are filled and no player wins, the game ends in a draw.

6.2. Swing-Based Board

In the Swing version, the 3×3 board can be represented by nine buttons.

```
+-----+-----+-----+
| Btn | Btn | Btn |
+-----+-----+-----+
| Btn | Btn | Btn |
+-----+-----+-----+
| Btn | Btn | Btn |
+-----+-----+-----+
```

Each button represents one cell. When the player clicks a button, the button text changes to X. Then the computer makes a move by changing another button text to O.

6.3. Suggested Score Calculation

The following scoring system is recommended:

Result	Score Change
Win	+10 points
Draw	+3 points
Lose	+0 points

Students may use a different score calculation, but it must be explained clearly in the README file, YouTube video, and classroom explanation.

7. Database Requirement

7.1. Database Management System

Students may use one of the following database systems:

1. MySQL,
2. PostgreSQL, or
3. SQL Server.

The application must connect to the selected database using JDBC.

7.2. One Table Only Rule

The database must contain **one table only**. This table stores user login data and game statistics. Students must use the database schema given by the lecturer.

7.3. Example Database Schema

The following schema is an example only. If the lecturer provides another schema, students must use the provided schema.

Table name: players

Columns:

- id
- username
- password
- wins
- losses
- draws
- score

7.4. Example SQL for MySQL

```
CREATE DATABASE game_project;
```

```
USE game_project;
```

```
CREATE TABLE players (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    username VARCHAR(50) NOT NULL UNIQUE,  
    password VARCHAR(100) NOT NULL,  
    wins INT DEFAULT 0,  
    losses INT DEFAULT 0,  
    draws INT DEFAULT 0,  
    score INT DEFAULT 0  
);
```

```
INSERT INTO players (username, password, wins, losses, draws, score)
```

```
VALUES ('student1', '12345', 0, 0, 0, 0);

INSERT INTO players (username, password, wins, losses, draws, score)
VALUES ('student2', '12345', 0, 0, 0, 0);

INSERT INTO players (username, password, wins, losses, draws, score)
VALUES ('student3', '12345', 0, 0, 0, 0);

INSERT INTO players (username, password, wins, losses, draws, score)
VALUES ('student4', '12345', 0, 0, 0, 0);

INSERT INTO players (username, password, wins, losses, draws, score)
VALUES ('student5', '12345', 0, 0, 0, 0);
```

7.5. Example SQL for PostgreSQL

```
CREATE DATABASE game_project;

CREATE TABLE players (
    id SERIAL PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(100) NOT NULL,
    wins INT DEFAULT 0,
    losses INT DEFAULT 0,
    draws INT DEFAULT 0,
    score INT DEFAULT 0
);

INSERT INTO players (username, password, wins, losses, draws, score)
VALUES ('student1', '12345', 0, 0, 0, 0);
```

7.6. Example SQL for SQL Server

```
CREATE DATABASE game_project;
GO

USE game_project;
GO

CREATE TABLE players (
    id INT IDENTITY(1,1) PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(100) NOT NULL,
```

```
wins INT DEFAULT 0,  
losses INT DEFAULT 0,  
draws INT DEFAULT 0,  
score INT DEFAULT 0  
);  
  
INSERT INTO players (username, password, wins, losses, draws, score)  
VALUES ('student1', '12345', 0, 0, 0, 0);
```

8. Suggested Class Structure

Students must follow the class structure provided by the lecturer. If the provided structure is slightly different from the table below, students must prioritize the lecturer's provided structure.

Class Name	Responsibility
Main	Starts the program and opens the Login Window.
DatabaseManager	Handles JDBC database connection.
Player	Stores player data such as id, username, wins, losses, draws, and score.
PlayerService	Handles login, retrieving player data, updating statistics, and retrieving Top 5 scorers.
GameBoard	Stores the game board data.
GameLogic	Handles move validation, winner checking, draw checking, and computer moves.
LoginFrame	Swing window for username and password input.
MainMenuFrame	Swing window for the main menu after login.
GameFrame	Swing window for playing the game.
StatisticsFrame	Swing window for showing personal statistics.
TopScorersFrame	Swing window for showing Top 5 scorers using a table.

8.1. Simple OOP Rules

Students must apply simple Object-Oriented Programming. The program must not be written entirely in one class.

1. Each class should have a clear responsibility.
2. GUI classes should handle display and user interaction.
3. Service classes should handle database operations.
4. Logic classes should handle game rules.
5. Model classes should store data.

6. Avoid duplicated code.
7. Use meaningful class names, method names, and variable names.

9. Program Flow

The recommended program flow is as follows:

1. Program starts from `Main.java`.
2. `Main.java` opens `LoginFrame`.
3. User enters username and password.
4. `LoginFrame` calls `PlayerService.login()`.
5. `PlayerService` checks login data from the database.
6. If login fails, `LoginFrame` shows an error message.
7. If login succeeds, `MainMenuFrame` is opened.
8. User selects one of the menu buttons:
 - (a) Start Game,
 - (b) View My Statistics,
 - (c) View Top 5 Scorers,
 - (d) Exit.
9. If the user starts the game, `GameFrame` is opened.
10. The game is played until win, lose, or draw.
11. After the game ends, statistics are updated in the database.
12. User can return to the main menu or exit the program.

10. Starter Code and Completion Clues

The lecturer has provided the starter code, especially for the GUI. Students must complete the missing parts marked with comments such as `TODO`, `COMPLETE THIS`, or similar markers.

The following examples show the type of code students are expected to complete. Students must adapt the code to the actual starter code provided by the lecturer.

10.1. Main Class

```
import javax.swing.SwingUtilities;

public class Main {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(new Runnable() {
            public void run() {
                LoginFrame loginFrame = new LoginFrame();
                loginFrame.setVisible(true);
            }
        });
    }
}
```

10.2. DatabaseManager Class

Students must complete the database URL, username, and password according to their own database configuration.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DatabaseManager {

    // TODO: Change these values based on your database
    configuration
    private static final String URL = "jdbc:mysql://localhost:3306/
game_project";
    private static final String USER = "root";
    private static final String PASSWORD = "";

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

For PostgreSQL, the URL may look like this:

```
jdbc:postgresql://localhost:5432/game_project
```

For SQL Server, the URL may look like this:

```
jdbc:sqlserver://localhost:1433;databaseName=game_project;encrypt=
true;trustServerCertificate=true;
```

10.3. Player Class

```
public class Player {
    private int id;
    private String username;
    private int wins;
    private int losses;
    private int draws;
    private int score;

    public Player(int id, String username, int wins, int losses,
                  int draws, int score) {
        this.id = id;
        this.username = username;
        this.wins = wins;
        this.losses = losses;
        this.draws = draws;
        this.score = score;
    }

    public int getId() {
        return id;
    }

    public String getUsername() {
        return username;
    }

    public int getWins() {
        return wins;
    }

    public int getLosses() {
        return losses;
    }

    public int getDraws() {
        return draws;
    }

    public int getScore() {
        return score;
    }
}
```

10.4. PlayerService Login Method

Students must complete the login method so that the GUI can check username and password from the database.

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.util.ArrayList;

public class PlayerService {

    public Player login(String username, String password) {
        String sql = "SELECT * FROM players WHERE username = ? AND
password = ?";

        try {
            Connection conn = DatabaseManager.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql);

            stmt.setString(1, username);
            stmt.setString(2, password);

            ResultSet rs = stmt.executeQuery();

            if (rs.next()) {
                int id = rs.getInt("id");
                String uname = rs.getString("username");
                int wins = rs.getInt("wins");
                int losses = rs.getInt("losses");
                int draws = rs.getInt("draws");
                int score = rs.getInt("score");

                return new Player(id, uname, wins, losses, draws,
score);
            }

            } catch (Exception e) {
                System.out.println("Login error: " + e.getMessage());
            }

            return null;
        }
    }
}
```

10.5. Updating Statistics

Students must complete the update method and call it after the game ends.

```
public void updateStatistics(Player player, String result) {
    int additionalScore = 0;
    String sql = "";

    if (result.equalsIgnoreCase("WIN")) {
        additionalScore = 10;
        sql = "UPDATE players SET wins = wins + 1, " +
            "score = score + ? WHERE id = ?";
    } else if (result.equalsIgnoreCase("LOSE")) {
        additionalScore = 0;
        sql = "UPDATE players SET losses = losses + 1, " +
            "score = score + ? WHERE id = ?";
    } else if (result.equalsIgnoreCase("DRAW")) {
        additionalScore = 3;
        sql = "UPDATE players SET draws = draws + 1, " +
            "score = score + ? WHERE id = ?";
    }

    try {
        Connection conn = DatabaseManager.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql);

        stmt.setInt(1, additionalScore);
        stmt.setInt(2, player.getId());

        stmt.executeUpdate();

    } catch (Exception e) {
        System.out.println("Update statistics error: " + e.
            getMessage());
    }
}
```

10.6. LoginFrame Event Handling

The GUI layout may already be provided. Students must complete the login button action.

```
import javax.swing.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
```



```
public class LoginFrame extends JFrame {
    private JTextField txtUsername;
    private JPasswordField txtPassword;
    private JButton btnLogin;
    private PlayerService playerService;

    public LoginFrame() {
        playerService = new PlayerService();

        // GUI layout is already provided by the lecturer.
        // Students only need to complete the missing logic.

        btnLogin.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                // TODO: Get username from txtUsername
                // TODO: Get password from txtPassword
                // TODO: Call playerService.login(username, password
            )

                // TODO: If login succeeds, open MainMenuFrame
                // TODO: If login fails, show JOptionPane error
                message
            }
        });
    }
}
```

Example completion clue:

```
String username = txtUsername.getText();
String password = new String(txtPassword.getPassword());

Player player = playerService.login(username, password);

if (player != null) {
    JOptionPane.showMessageDialog(this, "Login successful!");
    MainMenuFrame menuFrame = new MainMenuFrame(player);
    menuFrame.setVisible(true);
    this.dispose();
} else {
    JOptionPane.showMessageDialog(this, "Invalid username or
    password!");
}
```

10.7. MainMenuFrame Button Handling

```
public class MainMenuFrame extends JFrame {
    private Player currentPlayer;
    private JButton btnStartGame;
    private JButton btnStatistics;
    private JButton btnTopScorers;
    private JButton btnExit;

    public MainMenuFrame(Player player) {
        this.currentPlayer = player;

        // GUI layout is already provided.

        btnStartGame.addActionListener(e -> {
            GameFrame gameFrame = new GameFrame(currentPlayer);
            gameFrame.setVisible(true);
            this.dispose();
        });

        btnStatistics.addActionListener(e -> {
            StatisticsFrame statisticsFrame = new StatisticsFrame(
currentPlayer);
            statisticsFrame.setVisible(true);
        });

        btnTopScorers.addActionListener(e -> {
            TopScorersFrame topFrame = new TopScorersFrame();
            topFrame.setVisible(true);
        });

        btnExit.addActionListener(e -> {
            System.exit(0);
        });
    }
}
```

10.8. GameLogic Class

Students must complete the game logic. The GUI should call this class instead of putting all logic directly inside the frame.

```
public class GameLogic {
    private char[] board;

    public GameLogic() {
```

```
        board = new char[9];
        resetBoard();
    }

    public void resetBoard() {
        for (int i = 0; i < board.length; i++) {
            board[i] = ' ';
        }
    }

    public boolean makeMove(int index, char symbol) {
        if (index < 0 || index >= 9) {
            return false;
        }

        if (board[index] != ' ') {
            return false;
        }

        board[index] = symbol;
        return true;
    }

    public boolean checkWinner(char symbol) {
        int[][] patterns = {
            {0, 1, 2}, {3, 4, 5}, {6, 7, 8},
            {0, 3, 6}, {1, 4, 7}, {2, 5, 8},
            {0, 4, 8}, {2, 4, 6}
        };

        for (int i = 0; i < patterns.length; i++) {
            int a = patterns[i][0];
            int b = patterns[i][1];
            int c = patterns[i][2];

            if (board[a] == symbol && board[b] == symbol && board[c]
== symbol) {
                return true;
            }
        }

        return false;
    }

    public boolean isDraw() {
```

```
        for (int i = 0; i < board.length; i++) {
            if (board[i] == ' ') {
                return false;
            }
        }
        return true;
    }

    public int computerMove() {
        // TODO: Select an empty cell for computer move.
        // A simple random move is acceptable.
        return -1;
    }

    public char[] getBoard() {
        return board;
    }
}
```

10.9. GameFrame Button Handling

Students must complete the action listener for each game button.

```
public class GameFrame extends JFrame {
    private Player currentPlayer;
    private PlayerService playerService;
    private GameLogic gameLogic;
    private JButton[] buttons;
    private JLabel lblStatus;

    public GameFrame(Player player) {
        this.currentPlayer = player;
        this.playerService = new PlayerService();
        this.gameLogic = new GameLogic();

        // GUI layout is already provided.
        // Assume buttons[0] until buttons[8] represent the board
        cells.

        for (int i = 0; i < buttons.length; i++) {
            final int index = i;
            buttons[i].addActionListener(e -> handlePlayerMove(index
));
        }
    }
}
```

```
private void handlePlayerMove(int index) {
    // TODO: Make player move using gameLogic.makeMove(index, 'X
')
    // TODO: Update button text to X
    // TODO: Check whether player wins
    // TODO: Check draw
    // TODO: Generate computer move
    // TODO: Update computer button text to O
    // TODO: Check whether computer wins
    // TODO: Update database statistics after game ends
}

private void finishGame(String result) {
    playerService.updateStatistics(currentPlayer, result);
    JOptionPane.showMessageDialog(this, "Game result: " + result
);

    MainMenuFrame menuFrame = new MainMenuFrame(currentPlayer);
    menuFrame.setVisible(true);
    this.dispose();
}
}
```

10.10. TopScorersFrame Using JTable

The Top 5 scorers should be displayed using JTable. The data must be taken from the database.

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.util.ArrayList;

public class TopScorersFrame extends JFrame {
    private JTable table;
    private PlayerService playerService;

    public TopScorersFrame() {
        playerService = new PlayerService();

        String[] columns = {"Username", "Wins", "Losses", "Draws", "
Score"};
        DefaultTableModel model = new DefaultTableModel(columns, 0);

        // TODO: Get Top 5 players from playerService
    }
}
```

```
// TODO: Add each player data into the table model

table = new JTable(model);
add(new JScrollPane(table));
}
}
```

10.11. Top 5 Query

For MySQL and PostgreSQL:

```
SELECT username, wins, losses, draws, score
FROM players
ORDER BY score DESC, wins DESC
LIMIT 5;
```

For SQL Server:

```
SELECT TOP 5 username, wins, losses, draws, score
FROM players
ORDER BY score DESC, wins DESC;
```

11. Parts That Must Be Modified or Completed by Students

Students must clearly complete the missing parts of the program. The following checklist should be used:

No	Part	Student Task
1	DatabaseManager	Complete database URL, user-name, and password.
2	PlayerService.login()	Complete SQL query and return the correct Player object.
3	PlayerService.updateStatistics()	Update wins, losses, draws, and score after game result.
4	PlayerService.getTopFiveScorers()	Retrieve Top 5 players from database.
5	LoginFrame	Complete login button event handling.
6	MainMenuFrame	Complete navigation buttons.
7	GameLogic	Complete valid move, winner checking, draw checking, and computer move.
8	GameFrame	Connect game buttons with game logic and database update.
9	StatisticsFrame	Show personal statistics from database.
10	TopScorersFrame	Show Top 5 scorers using JTable.

12. Minimum Code Requirements

The submitted program must contain at least:

1. One main class.
2. One database connection class.
3. One player data class.
4. One service class for login and statistics.
5. One game logic class.
6. One Login Swing frame.
7. One Main Menu Swing frame.
8. One Game Swing frame.
9. One Statistics Swing frame or panel.
10. One Top 5 Scorers Swing frame or panel.

Students may add small helper methods if needed. However, the project must remain simple and understandable.

13. Development Steps

Students are strongly advised to develop the project in the following order:

1. Download or copy the starter GUI code provided by the lecturer.
2. Create the database and the required table.
3. Insert sample users into the database.
4. Add the JDBC driver to the Java project.
5. Configure `DatabaseManager`.
6. Test the database connection.
7. Complete the `Player` and `PlayerService` classes.
8. Test login directly before connecting it to the GUI.
9. Complete `LoginFrame` button handling.
10. Complete `MainMenuFrame` navigation.
11. Complete `GameLogic`.
12. Connect `GameFrame` buttons to `GameLogic`.
13. Update statistics after win, lose, or draw.
14. Complete the personal statistics display.
15. Complete the Top 5 scorers display using `JTable`.
16. Test the complete Swing application.
17. Upload the final source code to GitHub.
18. Record and upload the YouTube demonstration video.
19. Submit GitHub and YouTube links in the classroom system.

14. Testing Guide

Students must test the application before submission.

14.1. GUI Testing

No	Test Case	Expected Result
1	Open application	Login Window appears.
2	Click login with correct data	Main Menu Window appears.
3	Click login with wrong data	Error message appears.
4	Click Start Game	Game Window appears.
5	Click My Statistics	Statistics Window appears.
6	Click Top 5 Scorers	Top 5 table appears.
7	Click Exit	Application closes.

14.2. Game Testing

No	Test Case	Expected Result
1	Player clicks empty cell	Cell changes to X.
2	Player clicks occupied cell	Program rejects or ignores the move.
3	Player gets three symbols in a row	Program shows win result.
4	Computer gets three symbols in a row	Program shows lose result.
5	Board is full without winner	Program shows draw result.

14.3. Database Testing

No	Test Case	Expected Result
1	User wins a game	Wins and score increase in database.
2	User loses a game	Losses increase in database.
3	User gets a draw	Draws and score increase in database.
4	User opens Top 5 Scorers	Top 5 players are retrieved from database.

15. GitHub Submission Requirement

Students must upload the complete source code to GitHub.

The GitHub repository must contain:

1. Complete Java source code.
2. SQL file for creating the database table.
3. README file.

4. Screenshots of the running GUI.
5. Clear instructions on how to run the program.

15.1. Recommended GitHub Repository Structure

```
student-swing-game-project/  
|  
+-- src/  
|   +-- Main.java  
|   +-- DatabaseManager.java  
|   +-- Player.java  
|   +-- PlayerService.java  
|   +-- GameLogic.java  
|   +-- LoginFrame.java  
|   +-- MainMenuFrame.java  
|   +-- GameFrame.java  
|   +-- StatisticsFrame.java  
|   +-- TopScorersFrame.java  
|  
+-- database/  
|   +-- schema.sql  
|  
+-- screenshots/  
|   +-- login-window.png  
|   +-- game-window.png  
|   +-- top-scorers-window.png  
|  
+-- README.md
```

15.2. README File Content

The README file must include:

1. Project title.
2. Student name and student ID.
3. Class.
4. Description of the game.
5. Features of the application.
6. Database used.
7. How to create the database.

8. How to run the program.
9. Explanation of classes.
10. Screenshots of the Swing GUI.
11. GitHub repository link.
12. YouTube demonstration video link.

Example README outline:

```
# Simple Tic-Tac-Toe Game with Java Swing, Login, and Statistics

## Student Information
Name:
Student ID:
Class:

## Project Description
This project is a simple Tic-Tac-Toe game built using Java Swing.
The application includes login, game statistics, and Top 5 scorer
  feature.

## Features
- Login using database
- Play Tic-Tac-Toe using Swing GUI
- Record wins, losses, draws, and score
- Display personal statistics
- Display Top 5 scorers using JTable

## Database
Database used: MySQL / PostgreSQL / SQL Server

## How to Run
1. Create the database.
2. Import schema.sql.
3. Open the Java project.
4. Add JDBC driver.
5. Configure DatabaseManager.java.
6. Run Main.java.

## Class Explanation
Main:
DatabaseManager:
Player:
PlayerService:
```

```
GameLogic:
LoginFrame:
MainMenuFrame:
GameFrame:
StatisticsFrame:
TopScorersFrame:

## Screenshots
Add screenshots here.

## Video Link
YouTube:
```

16. YouTube Video Requirement

Students must create a demonstration video and upload it to YouTube.

The video must show:

1. Student introduction.
2. Short explanation of the project.
3. Explanation of the one-table database.
4. Explanation of the class structure.
5. Explanation of the parts completed or modified by the student.
6. Demonstration of the Swing Login Window.
7. Demonstration of successful and failed login.
8. Demonstration of playing the game.
9. Demonstration of win, lose, or draw result.
10. Demonstration that statistics are updated in the database.
11. Demonstration of Top 5 scorers using the GUI table.
12. Short conclusion.

16.1. Recommended Video Structure

Part	Content
Opening	Introduce name, student ID, and project title.
Project Overview	Explain what the application does.
Database Explanation	Show the one-table database structure.
GUI Explanation	Show Login Window, Main Menu, Game Window, Statistics, and Top 5 Scorers.
Code Explanation	Explain important classes and completed TODO parts.
Program Demonstration	Run the program and demonstrate all required features.
Closing	Mention GitHub repository and final remarks.

The video must clearly show the screen and voice explanation. Students must not only show the final application, but also explain how the program works.

17. Classroom Explanation

Each student must be ready to explain the project in class.

The explanation may include:

1. Why the student chose the game.
2. How the Swing GUI is structured.
3. How the login button works.
4. How the program connects to the database.
5. How game buttons are connected to the game logic.
6. How the program detects win, lose, and draw.
7. How the program updates wins, losses, draws, and score.
8. How the Top 5 scorers are displayed.
9. Which parts of the starter code were completed by the student.
10. What difficulties were found during development.

Students must be able to answer questions from the lecturer.

18. Submission

18.1. Submission Deadline

The submission deadline is:

Friday, 26 June 2026

18.2. Submission Items

Students must submit:

1. GitHub repository link.
2. YouTube video link.
3. Short project description.

The GitHub repository must be public or accessible to the lecturer.

18.3. Submission Format

Students may submit the links in the following format:

Name	:
Student ID	:
Class	:
Project Title:	
GitHub Link:	
YouTube Link:	

19. Assessment Criteria

The project will be assessed based on the following criteria:

Assessment Component	Weight
Swing GUI functionality and navigation	15%
Game functionality and correctness	15%
Login feature using database	10%
Database connection and one-table implementation	15%
Statistics recording: wins, losses, draws, and score	15%
Top 5 scorer feature using database and GUI table	10%
Simple Object-Oriented Programming and class separation	10%
Completion of required TODO or missing parts	5%
GitHub submission, README, and screenshots	3%
YouTube video and classroom explanation	2%
Total	100%

20. Academic Integrity

This is an individual project. Students must submit their own work.

Students are not allowed to:

1. Copy another student's complete project.
2. Submit a project created by another person.
3. Submit a project without understanding the code.
4. Use a database structure that violates the one-table requirement.
5. Submit only code without GitHub and YouTube links.
6. Submit a console-only version instead of the required Swing GUI version.

Students are allowed to:

1. Use the starter GUI code provided by the lecturer.
2. Discuss general programming concepts with classmates.
3. Read official documentation and tutorials.
4. Ask questions during consultation sessions.

However, every student must understand and explain their own submitted code.

21. Common Mistakes to Avoid

Students should avoid the following mistakes:

1. Putting all logic inside GUI frame classes.
2. Forgetting to connect Swing buttons to action listeners.
3. Forgetting to update the database after the game ends.
4. Displaying Top 5 scorers from hardcoded data instead of database data.
5. Creating more than one database table.
6. Forgetting to include the JDBC driver.
7. Using the wrong database username, password, or URL.
8. Not validating game moves.
9. Allowing players to click an occupied cell.
10. Not showing error messages when login fails.
11. Uploading incomplete code to GitHub.
12. Providing a private GitHub repository without access.
13. Uploading a video that does not explain the code.

22. Final Checklist

Before submission, students must check the following:

1. The program can run without error.
2. The Login Window appears correctly.
3. The login feature works using database data.
4. The Main Menu Window appears after successful login.
5. The Game Window can be opened.
6. The game can detect win, lose, and draw.
7. The statistics are updated in the database.
8. The My Statistics Window displays correct data.
9. The Top 5 Scorers Window displays data from the database.

10. The program uses only one database table.
11. The program is divided into several classes.
12. The required TODO parts are completed.
13. The GitHub repository contains complete source code.
14. The GitHub repository contains the SQL schema file.
15. The README file is complete.
16. The repository contains GUI screenshots.
17. The YouTube video clearly demonstrates and explains the program.
18. The GitHub and YouTube links are submitted before the deadline.

23. Closing Statement

This small project is an opportunity for students to demonstrate their understanding of programming fundamentals in a complete Java Swing application. Students are expected to understand how GUI components, event handling, game logic, simple classes, and database operations work together.

The most important point is not only that the program runs, but also that students understand how and why the program works.